

A Dynamic-Causal Lens of Stock Interactions: Graph Modeling From Symbolic Movement Patterns

Haotian Liu , Bowen Hu , Yadong Zhou , *Member, IEEE*, and Yuxun Zhou 

Abstract—In modern quantitative finance and portfolio-based investment, modeling latent interactions among stocks is paramount for prediction and decision-making on profit, risk management, hedging, etc. While previous studies have constructed complex stock graphs for applying sophisticated variants of graph neural networks (GNNs), existing graph modeling approaches still face two limitations: 1) Correlation-based statistical relationships fail to unveil nuanced stock interactions effectively and determine directional influence. 2) Rigid and static graphs overlook the evolving graph structure of stocks in volatile financial systems. In this paper, we propose a dynamic-causal graph neural network (DC-GNN) to discover causal interactions from the non-stationary price time series and dynamically model graph structures for stock movement prediction. More specifically, we identify the pattern prototypes of all directed stock pairs from long-term price movement knowledge to quantify their causal interactions. These prototypes capture the pattern-to-pattern correspondence across time series based on symbolic dynamics. By inferring real-time stock networks from the prototypes, we encapsulate neighbor-induced causal impacts within heterogeneous edges to model bullish, bearish, and neutral effects among stocks. Extensive experiments conducted on real-world trading data demonstrate the superiority of the proposed framework over various state-of-the-art baselines and its effectiveness, robustness, and interpretability in Fintech.

Index Terms—Dynamic graph modeling, graph neural networks, causality, stock movement prediction.

I. INTRODUCTION

IN TODAY'S fast-paced and interconnected financial markets, accurately forecasting stock movements holds profound implications for investors, financial institutions, and policy-makers. With trillions of dollars traded daily on global stock exchanges, even the slightest deviations from expected price trends can reverberate throughout investment portfolios, market stability, and broader economic growth. Consequently, the pursuit of reliable methods for the prediction of stock movements has emerged as a central imperative in the practical applications of Fintech.

Received 23 April 2025; revised 6 November 2025; accepted 24 November 2025. Date of publication 28 November 2025; date of current version 30 December 2025. The work was supported by the National Natural Science Foundation of China under Grant 62272373. Recommended for acceptance by S. Whang. (*Corresponding author: Yadong Zhou.*)

Haotian Liu, Bowen Hu, and Yadong Zhou are with the Ministry of Education Key Laboratory for Intelligent Networks and Network Security, Faculty of Electronic and Information Engineering, Xi'an Jiaotong University, Xi'an 710049, China (e-mail: ydzhou@xjtu.edu.cn).

Yuxun Zhou is with the Department of Electrical Engineering and Computer Sciences, University of California at Berkeley, Berkeley, CA 94720 USA.

Digital Object Identifier 10.1109/TKDE.2025.3638410

As technical analysis evolves, traditional sequential learning methods rely on various variants of recurrent neural networks (RNNs) to capture the underlying time-varying patterns from multiple time steps, wherein each stock is represented by its multivariate time-series information, e.g., opening price, closing price, and volume [1], [2], [3], [4]. Nevertheless, given that all stocks belong to the same trading system, the variation of one entity is inevitably intertwined with its peer entities. In particular, ample evidence exists, indicating that this interdependence often manifests as a lead-lag effect, where the price movement of one stock shows a delayed reaction to the price changes in others [5]. Recent explorations employ graph modeling by conceptualizing the stock market as a complex network, where stocks are treated as nodes and edges are defined by their relations. Then, graph neural networks (GNNs) are applied to aggregate stock influences to predict price movements, which offer a promising avenue for modeling interactions among entities [6]. But in the financial scenario, no ground-truth physical linkages among stocks are available for graph modeling. Researchers have to rely on statistical correlations or prior knowledge to construct heterogeneous graphs [7], hypergraphs [8], [9], or knowledge graphs [9], [10] tailored to model this market-wide phenomenon.

In literature, there exist various predefined stock relationships directly modeled by time-series correlation [7], industry category [10], supply chain [11], shareholding [9], etc. However, contradictions emerge in the stock market where variables (e.g., prices) exhibit positive coupling at certain times but appear unrelated or even negatively coupled at other times, depending on the financial system's state [12]. For example, the stock market often exhibits time-varying momentum spillover effect¹ depending on the economic cycle, assets liquidity, and macro regulation, causing rapid changes in the correlations between two stocks [4]. Such state-sensitive behavior is a defining hallmark of dynamic nonlinear systems. This suggests that the inter-node interactions often evolve over time, and static graphs may be suboptimal for the downstream tasks. Traditional hard-coded graphs struggle to grasp the temporal dynamics and evolving hidden structures inherent in the financial market. Although graph attention networks (GATs) [13], [14] offer some insight into internal dynamics [6], [10], [15], they essentially rely on the adaptive weighting of neighboring nodes, rather than on modeling temporal dynamics or capturing long-term dependencies.

¹ Momentum spillovers among stocks refer to the phenomenon where the past prices of one stock influence the future prices of its peer stocks as a result of factor movement, lead-lag effect, portfolio rebalancing, etc.

On the other hand, since ephemeral or “mirage” correlations are common in multivariate time-series analysis, graphs constructed by gauging inter-stock interactions with correlation are known to be incomplete, noisy, and even misleading. Furthermore, GNNs compute node embeddings by recursively aggregating information from neighborhoods. This iterative process can have cascading effects: small amounts of noise in the graph can be propagated to adjacent nodes, potentially distorting their embeddings and leading to inaccurate predictions. Several studies try to learn an optimized graph using GNNs and offer some topological dynamics [14], [16], [17]. Essentially, their kernel functions still measure the similarity between nodes, which neither overcome the limitation of correlation nor determine edge directionality. The robust notion is that of causality in time series, which has been approached via methods such as Granger causality [18], cointegration [19], transfer entropy [20], convergent cross mapping (CCM) [21], and pattern causality [22], [23]. Yet, few works have incorporated causality when inferring stock relationships, and more importantly, none of the aforementioned causal inference methods provide insight regarding the dynamics of stock interactions. How to effectively integrate causality into dynamic graph modeling is still an unresolved issue. Furthermore, even if there is a potential causal relation, it does not necessarily result in sustained and stable explicit interactions. This implies that the causal mechanisms between financial time series may be more complex, and existing single-valued causal effect estimation methods [12], [24] would be insufficient, if not impossible, to describe more complex causal mechanisms.

To discover both causality and dynamics within the stock interactions, we propose a framework named dynamic-causal graph neural network (DC-GNN) for stock movement prediction,² which combines a dynamic-causal graph modeling method and a tailored GNN variant. In this work, these two attributes, i.e., dynamics and causality, are combined as *state-dependent causality*, which is manifested in that only when a stock’s price time series exhibits a particular pattern, will it be triggered and exert a causal effect on related stocks, rather than unconditionally. Specifically, causality is no longer quantified as a static constant between two stocks but as a pattern prototype matrix between their symbolic price movement patterns, which is based on convergent cross mapping (CCM) [21] and pattern causality [22]. By retrieving information from all prototypes according to the latest historical price trends, our method dynamically models a real-time stock network wherein heterogeneous edges represent neighbor-induced causal influences that consist of bullish, bearish, and neutral effects. As the observable time steps increase, the identified pattern prototypes can be gradually updated to adapt to evolving market contexts. Based on the inferred stock networks, our framework filters out weak causality and aggregates dynamic-causal effects from the neighborhood using GNNs. Our main contributions are summarized as follows:

- 1) We recognize the matrix-valued causality of directed stock pairs for their graph structure modeling. By leveraging symbolic dynamics, our method discerns between

positive, negative, and neutral causality to model the underlying inducements heterogeneously.

- 2) We dynamically infer the time-varying stock networks from the symbolic price patterns. These networks seamlessly integrate temporal forward causality into the topology evolution and proactively capture lead-lag effects.
- 3) Extensive experiments conducted on real-world trading data consistently demonstrate that our framework outperforms a series of state-of-the-art baselines and maintains sufficient effectiveness, robustness, and interpretability.

II. RELATED WORK

This research aims to integrate time-series causal discovery methods into dynamic graph modeling for stock movement prediction. In this section, we briefly review the related work on stock movement prediction, graph modeling, and causal inference.

A. Stock Movement Prediction

1) *Time-Series-Based Methods*: Traditional methodologies for stock movement prediction often focus on technical analysis which involves scrutinizing time-series attributes. Typically, these methods capture sequential hidden representations by employing recurrent neural networks (RNNs) [25] for encoding temporal information. A common strategy for predicting the price movement of individual stocks is to collect price and volume signals or to construct a series of indicators as inputs to RNN variants to extract technical features [1], [3], [4], [7], [10]. Some studies have also explored integrating diverse market indicators and fundamental factors, e.g., messages from social media [26], financial news [15], and futures markets [27], to improve model performance. For example, PEN [2] amalgamates multimodal features, including technical indicators and textual news, to extract investment insights. Digger-Guider [28] extracts local and detailed features from the high-frequency data and captures global tendency features to overcome the noise. However, an underlying assumption in these studies is that market signals for all stocks are mutually exclusive, which does not align with observations in real-world scenarios.

2) *Graph-Based Methods*: Since all entities operate within the same financial system, the trajectory of each stock’s price is intricately influenced by its peer stocks [6], [29], [30]. This interconnectedness is often represented as a stock network by defining different stock relationships, such as industry category [11], supply chain [10], price correlation [7], earning calls [31], and investment bank [32]. As a next step, various forms of graph neural networks (GNNs) are applied to aggregate interactions from the neighborhood [7], [33]. Nevertheless, a notable limitation of these approaches is the over-reliance on static graph structures throughout the entire pipeline. For instance, ESTIMATE [8] generates hypergraphs based on industry classifications to capture non-pairwise stock correlation. PTER [9] introduces hypernetworks and forms a relaxed weight-sharing paradigm to address distribution shift in macro market scenarios. These techniques define stock interactions as hard-coded graphs, lacking the adaptability to dynamic market conditions.

²The code is accessible at <https://github.com/rlgain/DCGNN>

Recent efforts have been made to address these limitations by modeling time-varying relationships among stocks [34]. For example, FinGAT [35] and PA-TMM [15] dynamically learn the latent interactions among stocks and sectors using GATs. LSR-IGRU [36] models long-term stock relationships via secondary industry information and captures short-term dynamics through overnight return-driven relationship networks. MGGAL [37] builds multi-scale temporal return graphs, i.e., minute-level, daily, weekly, to discover dynamic cross-frequency stock interactions. DTML [38] integrates temporal, global, and inter-stock contexts to capture asymmetric and dynamic correlations among stocks. RT-GCN [29] and MATCC [39] model the temporal structure of stocks by mining cross-time correlations. MDGNN [32] integrates multi-relational dynamic graphs with a Transformer architecture to effectively model complex inter-stock relationships and their temporal evolution. However, these similarity- or correlation-based statistical relationships may not sufficiently capture the actual stock interactions.

B. Causal Inference

Since correlation does not imply causation, correctly inferring causality has garnered significant attention. It helps reason about a dynamic system, anticipate its behavior in previously unseen conditions, design changes to achieve some objective, or synthesize new systems with desirable behaviors. As an example, in the financial system, causal inference could allow predicting whether a surging stock will have a positive impact on related stocks' prices.

1) *Causal Effect Estimation*: A central problem in causal inference is to quantify the strength of causal effects between treatments (or interventions) and outcomes under the premise of a known or assumed causal structure. In general, this task requires conducting randomized controlled trials (RCTs) which involve dividing all samples or individuals into a treatment group and a control group. By comparing the differences in outcomes between the two groups, the causal effect of the treatment can be determined. However, real-world dynamic systems are often only observable or costly controllable, such as ecosystems, financial systems, and social networks. Data collected from less controlled environments than, for instance, clinical trials pose many challenges for identifying causal dependencies when only observational data is available. If no intervention is possible, some causal configurations cannot be identified. To quantify the causal effect of intervention from Big Data, researchers develop various frameworks, including the potential outcome framework [40], [41], uplift modeling [42], and the do-calculus in structural causal model (SCM) [24], [43].

2) *Causal Discovery*: Beyond causal effect estimation, another line of causal inference, namely causal discovery (or causal structure learning), aims to uncover causal relationships by constructing a causal graph. This task associates a set of observed objects, modeled as random variables, with the vertices of directed acyclic graphs (DAGs, also known as Bayesian networks), to estimate the causal structure. Representative methods can be divided into three categories, including constraint-based models [44], score-based models [45], and function-based

models [46]. These methods are all contingent on specific statistical assumptions, and both PC algorithm (named after its authors, Peter and Clark) [44] and greedy equivalence search (GES) [45] may also produce undirected edges. To avoid any structural assumptions on the graph, gradient-based optimization methods, such as NOTEARS [47], have been introduced, leveraging neural networks to construct causal graphs.

3) *Causal Discovery for Time Series*: Actually, causal effect estimation methods rely on *independent and identically distributed* (i.i.d.) data and static treatments, while causal discovery methods often assume acyclic, structure-invariant relationships. These assumptions often break down in time-series tasks due to temporal dependencies, feedback loops, and non-stationarity. Granger causality [18] provided the first significant framework to infer causal dependencies from time series. Relying on predictability between dynamic systems, it was extended to account for different limitations, such as nonlinearity [48] or instantaneous relationships [49]. However, the assumption of separability, i.e., information about causative factors is not contained within the caused variable itself, often leads to limitations in the Granger causality paradigm, especially in more complex scenarios such as weakly coupled nonlinear dynamical systems. To this end, convergent cross mapping (CCM) [21] was introduced, which relies on nonlinear state space reconstruction. The intuition is that if a variable X causally influences Y , then the reconstructed manifold of Y should contain information about the states of X . By cross-mapping between reconstructed manifolds, CCM can identify bidirectional and nonlinear causal relationships. Recent research has expanded upon CCM to overcome specific challenges, including synchrony [50], non-stationarity [22], and confounding cases [51]. Some of these works leverage Gaussian processes to reconstruct noise-robust CCM [52], and others incorporate latent variable models [53] or deep learning [54] into CCM by introducing Neural ODEs (ordinary differential equations). Nevertheless, few studies have extended these methods to the domain of dynamic graph modeling, and a framework of causal discovery for inferring dynamic interactions remains largely unexplored.

III. PRELIMINARIES

In this section, we introduce technical indicators as input features of stocks, provide a formal definition of stock movement prediction, and discuss the foundational definitions of causality in time series.

A. Technical Indicators

In quantitative finance, technical indicators are mathematical calculations based on historical price, volume, and other factors, used to forecast future price movements of financial assets and estimate market sentiment [4]. In this study, basic trading indicators consist of the highest and lowest prices, opening and closing prices, trade volume, and their rankings over different periods, such as 5 days, 20 days, and 60 days. Furthermore, we augment the technical data with additional indicators, including moving average indicators, momentum indicators, volatility indicators, and volume indicators. To mitigate the impact of different price

levels of different stocks, all these indicators (excluding rank values) are standardized for each stock, which are treated as original temporal features.

B. Problem Definition

The task of stock movement prediction is defined as a binary classification problem with two classes: rise and fall [2], [10]. Given a set $\mathcal{V} = \{1, \dots, N\}$ of entities (stocks), the movement $m_{v_i}^t$ of $v_i \in \mathcal{V}$ on day t can be represented as binary

$$m_{v_i}^t = \begin{cases} 1, & c_{v_i}^t > c_{v_i}^{t-1} \\ 0, & c_{v_i}^t \leq c_{v_i}^{t-1} \end{cases} \quad (1)$$

where $c_{v_i}^t$ and $c_{v_i}^{t-1}$ correspond to the closing prices of stock v_i on day t and the previous day $t-1$, respectively. Formally, at time L , given technical indicators (historical features) with T_f observation days for all stocks, this task aims to predict the movement $\hat{m}_{v_i}^{L+1}$ for each stock on the upcoming trading day $L+1$.

C. Groundwork for Causality in Time Series

Here we lay the groundwork for understanding causality within dynamic systems, such as the stock market, drawing upon the framework of time-delay embedding theory [21], [55], [56], [57].

Consider a dynamic process ϕ defining the temporal evolution of points in a state space, with trajectories converging to a d_M -dimensional manifold $\mathcal{M}$ such that $\phi(\cdot) : \mathcal{M} \rightarrow \mathcal{M}$, where $\mathcal{M}^{t+\tau} = \phi(\mathcal{M}^t)$ and $\mathcal{M}^t$ represents the point on the manifold at time t . Let $\chi(\cdot)$ denote an observation function of ϕ , mapping points on $\mathcal{M}$ to a real-valued scalar, i.e., $\chi(\cdot) : \mathcal{M} \rightarrow \mathbb{R}$. Then, there exists a corresponding time series denoted as $\mathbf{X} = [\mathbf{X}^1, \dots, \mathbf{X}^L]$ that tracks the trajectory of points in $\mathcal{M}$ mapped to a sequence of real numbers, where $\mathbf{X}^t = \chi(\mathcal{M}^t)$. The size of the observation window is L .

1) *Shadow Manifold*: A delay embedding (or lagged-coordinate embedding) Φ with a time lag $\tau \in \mathbb{N}^+$ uses E time-lagged values of $\mathbf{X}$ as coordinate axes or dimensions to reconstruct a shadow attractor manifold $\mathcal{M}_X = \Phi^{E,\tau}(\mathcal{M})$, where each point is an E -dimensional vector: $\mathcal{M}_X^t = [\mathbf{X}^t, \mathbf{X}^{t-\tau}, \dots, \mathbf{X}^{t-(E-1)\tau}]$. In short, such a process uses a sliding time window with fixed size E to observe one time series in a forward direction. Each time series segment within the window can be regarded as a point in an E -dimensional shadow manifold. Takens' Theorem [57] ensures that $\mathcal{M}_X$ is a diffeomorphic reconstruction of the original attractor manifold $\mathcal{M}$. This implies that the original time series can theoretically be reconstructed from the delay embedding [21], [53], [55].

2) *Discerning Directed Causality*: To clarify how to determine causality using Takens' Theorem, consider a manifold $\mathcal{M}_Z$ of the joint dynamical system Z , formed by concatenating $\mathbf{X}^t$ and $\mathbf{Y}^t$, where $\mathbf{Y}^t = \gamma(\mathcal{M}^t)$ is analogous to $\mathbf{X}^t$. By employing delay embeddings, we construct two shadow manifolds $\mathcal{M}_X$ and $\mathcal{M}_Y$. Now, if X unidirectionally causes Y , i.e., Y does not cause X , it means that X is part of an autonomous dynamic system, while Y is part of a larger one encompassing X . Thus the manifold of Y coincides with that of the joint dynamical

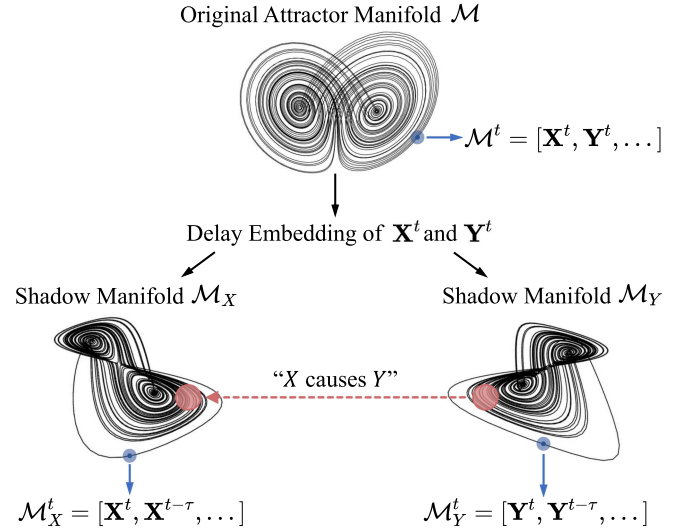


Fig. 1. Examples of an original attractor manifold ($\mathcal{M}$) and two reconstructed shadow manifolds ($\mathcal{M}_X$ and $\mathcal{M}_Y$). If X causes Y , there exists a continuous map from $\mathcal{M}_Y$ to $\mathcal{M}_X$.

system Z , whereas the manifold of X is only a subset thereof. According to Takens' Theorem, it is theoretically feasible to recover the original $\mathcal{M}_Z$ from $\mathcal{M}_Y$ and hence, by extension, recover $\mathcal{M}_X$ from $\mathcal{M}_Y$ [53]. However, it is generally impossible to recover $\mathcal{M}_Y$ from $\mathcal{M}_X$ in reverse.

3) *Convergent Cross Mapping*: Based on simplex projection, convergent cross mapping (CCM) [21] uses the reconstructed manifold for kernel density estimation, which enables the inference of causality in dynamic systems. To quantify the influence from X to Y , CCM evaluates how well local neighborhoods on $\mathcal{M}_Y$ correspond to contemporaneous local neighborhoods on $\mathcal{M}_X$, as depicted in Fig. 1. Specifically, it utilizes the $E+1$ nearest neighbors³ of each point on $\mathcal{M}_Y$ to estimate contemporaneous values of X . As $\mathcal{M}_Y$ is diffeomorphic to $\mathcal{M}$, estimates of X converge as L approaches infinity. Directed causality is thereby demonstrated by estimation precision that rises with L and eventually reaches a plateau.

However, CCM often proves ineffective for financial time series due to the non-stationarity and high volatility of different price levels [22], [23]. As the window size L increases, the shadow manifold typically fails to fill in, and the distances among the nearest neighbors fail to shrink. Eventually, estimates of X may not converge to the true values of X , which makes it challenging to test whether there is a correspondence between states on $\mathcal{M}_X$ and states on $\mathcal{M}_Y$. To overcome this challenge in finance, we propose a method for inferring dynamic and causal relationships among stocks based on symbolic dynamics, which focuses on relative price volatility rather than absolute price levels.

IV. METHOD

In this section, we present the framework of the dynamic-causal graph neural network (DC-GNN), focusing on the

³ $E+1$ is the minimum number of points necessary for a bounding simplex in an E -dimensional space.

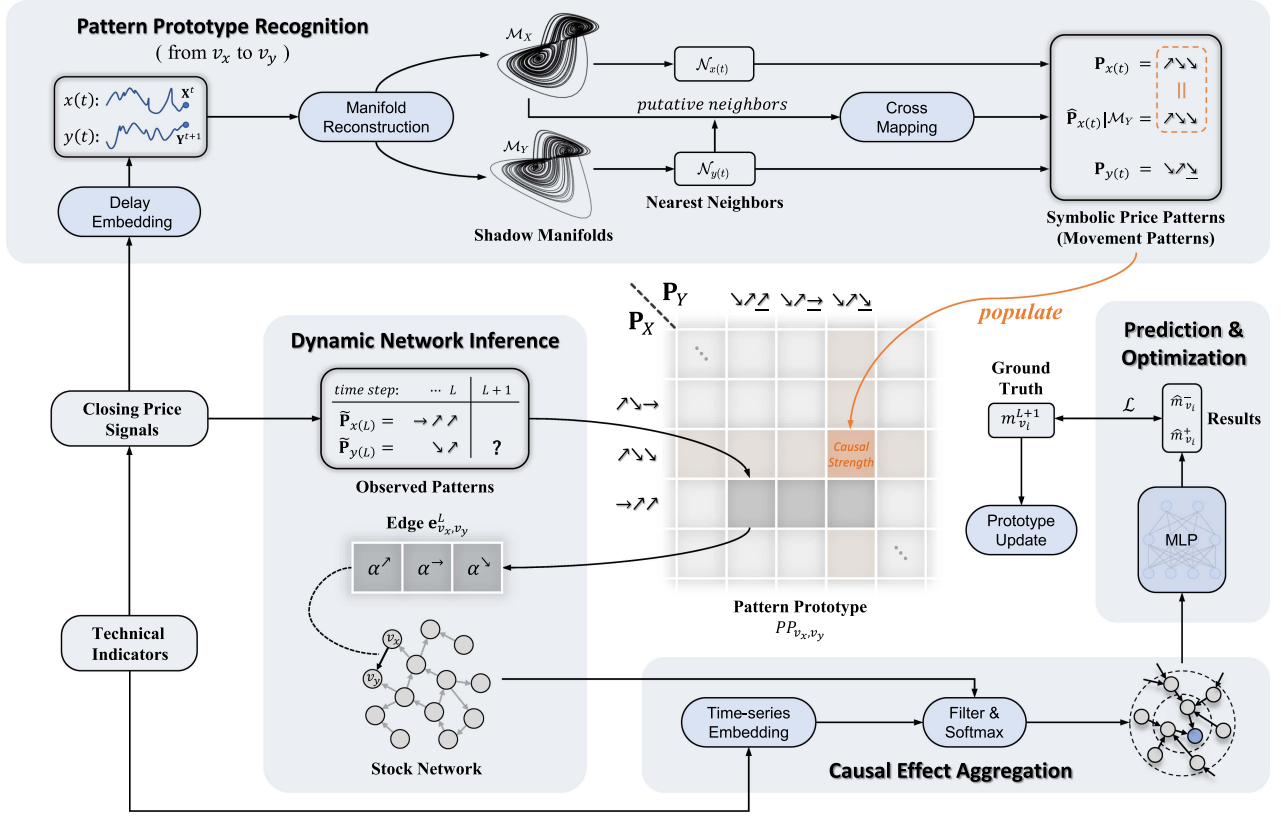


Fig. 2. Framework of the proposed dynamic-causal graph neural network (DC-GNN) for stock movement prediction.

dynamic-causal interactions of entities. As illustrated in Fig. 2, DC-GNN consists of four main components: a) *Pattern prototype recognition*, which quantifies the causality of each directed stock pair to fill in pattern prototype matrices. b) *Dynamic network inference*, which constructs real-time stock networks by inferring the heterogeneous edges. c) *Causal effect aggregation*, which integrates multi-hop causal influences to model stock interactions. d) *Prediction and optimization*, responsible for estimating price movements and learning all parameters.

A. Pattern Prototype Recognition (PPR)

We commence by uncovering and understanding latent causal relationships between different stocks. Specifically, we propose pattern prototype recognition to quantify the matrix-valued causality rather than a scalar to elucidate how the past symbolic price movement patterns in one stock cause the future behavior of another. Our method follows pattern causality [22], [23], which leverages symbolic dynamics for causal discovery in time series. Nevertheless, pattern causality overlooks the time-varying nature of financial markets, rendering it insufficient for dynamically inferring stock networks. Here, we integrate temporal forward causality into dynamic structures, facilitating real-time estimation of the effects of directed interventions on the target node. These effects manifest as bullish, neutral, or bearish inducements in finance.

Given two stocks $v_x, v_y \in \mathcal{V}$, we consider their historical closing price signals with length L , $\mathbf{X} = [\mathbf{X}^1, \dots, \mathbf{X}^L]$

and $\mathbf{Y} = [\mathbf{Y}^1, \dots, \mathbf{Y}^L]$, to quantify the causality from v_x to v_y . For determining the forward causality on a temporal scale, i.e., how current v_x influences future v_y , we shift the time series of $\mathbf{Y}$ forward by one time step when constructing the E -dimensional lagged-coordinate vectors. Specifically, we define $y(t) = [\mathbf{Y}^{t+1}, \mathbf{Y}^{t+1-\tau}, \dots, \mathbf{Y}^{t+1-(E-1)\tau}] = [y_t^1, \dots, y_t^E]$ with $\tau \in \mathbb{N}^+$ being the time lag, while $x(t) = [\mathbf{X}^t, \mathbf{X}^{t-\tau}, \dots, \mathbf{X}^{t-(E-1)\tau}] = [x_t^1, \dots, x_t^E]$. The sets of vectors $y(t)$ and vectors $x(t)$ constitute the reconstructed shadow manifolds $\mathcal{M}_Y$ and $\mathcal{M}_X$, respectively. We consider points from $t = (E-1)\tau$ to $t = L-1$ on both manifolds for pattern prototype recognition.

As stated in Section III-C, to calculate the directed causality from v_x to v_y , for each point on $\mathcal{M}_Y$, $y(t)$, we find its $E+1$ nearest neighbors, which is the minimum number of points needed for a bounding simplex in an E -dimensional space. From these neighbors we need to keep three pieces of information: 1) their time indices, 2) their distances from $y(t)$, and 3) their temporal return patterns (as described in the following). As a next step, the aforementioned pieces of information are used to estimate the average contemporaneous pattern of $x(t)$. For a causal relationship to be recognized without the influence of absolute price values, we begin by defining the temporal return pattern of $y(t)$ as follows:

$$\mathbf{r}_{y(t)} = \left[\frac{y_t^{E-1} - y_t^E}{y_t^E}, \dots, \frac{y_t^1 - y_t^2}{y_t^2} \right] \quad (2)$$

where $\mathbf{r}_{y(t)} \in \mathbb{R}^{E-1}$. To determine the commonality of patterns of v_y in scenarios similar to that at time t , we identify the $E+1$ nearest neighbors' return patterns of $y(t)$ and denote their time indices as $\mathcal{N}_{y(t)} = \{t_1, \dots, t_{E+1}\}$. These neighbors are used to generate the average pattern (movement signature) $\mathbf{P}_{y(t)}$ from a locally weighted mean of their return patterns, as a representation of the dominant pattern in a spatial-temporal neighborhood on $\mathcal{M}_Y$

$$\mathbf{P}_{y(t)} = \text{signature}_\theta \left(\sum_{t_i \in \mathcal{N}_{y(t)}} w_y(t, t_i) \mathbf{r}_{y(t_i)} \right) \quad (3)$$

where

$$w_y(t, t_i) = \frac{\exp(-d(y(t), y(t_i)))}{\sum_{t_j \in \mathcal{N}_{y(t)}} \exp(-d(y(t), y(t_j)))}. \quad (4)$$

We derive $\mathbf{P}_{x(t)}$ similarly. Here, $w_y(t, t_i)$ represents the weighting of each return pattern and $d(\cdot, \cdot)$ calculates the distance between two vectors, e.g., euclidean distance or Manhattan distance. The function $\text{signature}_\theta(\cdot)$ converts a vector to a symbolic price pattern (movement pattern), each element of which is represented as “ $\nearrow$ ”, “ $\rightarrow$ ”, or “ $\searrow$ ”. The symbolization threshold θ is used to determine whether the price volatility exhibits a significant increase or decrease, i.e., if the absolute value of a certain return does not exceed θ , it is considered to be relatively stationary and is symbolically mapped to “ $\rightarrow$ ”. For example, if $\theta = 0.05$ and $E = 5$, $\text{signature}([0.2, 0.01, -0.02, -0.1]) = [\nearrow, \rightarrow, \rightarrow, \searrow]$.

Then, to estimate a cross-mapped pattern of v_x at time t , denoted by $\hat{\mathbf{P}}_{x(t)}|\mathcal{M}_Y$, we locate the contemporaneous lagged-coordinate vector on $\mathcal{M}_Y$ and find its $E+1$ nearest neighbors that are used to identify the putative neighbors on $\mathcal{M}_X$ as follows [21], [22], [23], [53]:

$$\hat{\mathbf{P}}_{x(t)}|\mathcal{M}_Y = \text{signature}_\theta \left(\sum_{t_i \in \mathcal{N}_{y(t)}} w_y(t, t_i) \mathbf{r}_{x(t_i)} \right). \quad (5)$$

Finally, the real average pattern $\mathbf{P}_{x(t)}$ from the actual nearest neighbors $\mathcal{N}_{x(t)}$ of $x(t)$ is used as the ground-truth to verify the estimation from (5). We repeat this procedure for points from $t = (E-1)\tau$ to $t = L-1$, retaining for each possible pattern $\mathbf{P}_{y(t)}$ the weighted percentage of the occasions that the contemporaneous estimated dominant pattern $\hat{\mathbf{P}}_{x(t)}|\mathcal{M}_Y$ matches the real dominant pattern $\mathbf{P}_{x(t)}$. By this procedure, we populate the pattern-to-pattern causal prototype matrix $PP_{v_x, v_y} \in \mathbb{R}^{3^{E-1} \times 3^{E-1}}$ to quantify the causal influence from pattern $\mathbf{P}_X$ to pattern $\mathbf{P}_Y$ (which both are certain permutations with length $E-1$ of elements “ $\nearrow$ ”, “ $\rightarrow$ ”, and “ $\searrow$ ”)

$$PP_{v_x, v_y}[\mathbf{P}_X, \mathbf{P}_Y] = \frac{1}{|\mathcal{T}_{\mathbf{P}_X}|} \sum_{t \in \mathcal{T}_{\mathbf{P}_X}} \mathbf{I}(\mathbf{P}_{x(t)}, \mathbf{P}_{y(t)}) \quad (6)$$

where $\mathbf{P}_X, \mathbf{P}_Y \in \{(s_1, s_2, \dots, s_{E-1}) | s_i \in \{\nearrow, \rightarrow, \searrow\}\}$, and the strength of causality is determined by examining whether

Algorithm 1: Pattern Prototype Recognition.

Input : Closing price series $\mathbf{X}, \mathbf{Y}$ of stocks v_x, v_y with length L ; Sliding time window size E ; Time lag τ ; Threshold θ .

Output : Pattern prototype matrix PP_{v_x, v_y} .

- 1 **Step 1: Reconstruct Shadow Manifolds.**
- 2 **for** $t \leftarrow (E-1)\tau$ **to** $L-1$ **do**
- 3 $x(t) \leftarrow [\mathbf{X}^t, \mathbf{X}^{t-\tau}, \dots, \mathbf{X}^{t-(E-1)\tau}]$;
- 4 $y(t) \leftarrow [\mathbf{Y}^{t+1}, \mathbf{Y}^{t+1-\tau}, \dots, \mathbf{Y}^{t+1-(E-1)\tau}]$;
- 5 $\mathcal{M}_X \leftarrow \{x(t)\}, \mathcal{M}_Y \leftarrow \{y(t)\}$;
- 6 **Step 2: Symbolize Local Movement Patterns.**
- 7 **foreach** $y(t)$ **in** $\mathcal{M}_Y$ **do**
- 8 Locate the $E+1$ nearest neighbors of $y(t)$ on $\mathcal{M}_Y$ as $\mathcal{N}_{y(t)}$;
- 9 Compute the return series of $y(t)$ as $\mathbf{r}_{y(t)}$;
- 10 Symbolize the locally weighted mean of $\mathbf{r}_{y(t)}$ by threshold θ as $\mathbf{P}_{y(t)}$ using $\mathcal{N}_{y(t)}$;
- 11 Similarly derive $\mathbf{P}_{x(t)}$ for each point $x(t) \in \mathcal{M}_X$;
- 12 **Step 3: Estimate Cross Mapping Patterns.**
- 13 **foreach** $x(t)$ **in** $\mathcal{M}_X$ **do**
- 14 Locate the $E+1$ contemporaneous nearest neighbors of $x(t)$ on $\mathcal{M}_Y$ as $\mathcal{N}_{x(t)}|\mathcal{M}_Y$;
- 15 Symbolize the locally weighted mean of $\mathbf{r}_{x(t)}$ by threshold θ as $\hat{\mathbf{P}}_{x(t)}|\mathcal{M}_Y$ using $\mathcal{N}_{x(t)}|\mathcal{M}_Y$;
- 16 **Step 4: Populate Pattern Prototype.**
- 17 Initialize PP_{v_x, v_y} as zero matrix of size $3^{E-1} \times 3^{E-1}$;
- 18 $\mathcal{P}_{E-1} \leftarrow \{(s_1, s_2, \dots, s_{E-1}) | s_i \in \{\nearrow, \rightarrow, \searrow\}\}$;
- 19 **foreach** $\mathbf{P}_X$ **in** $\mathcal{P}_{E-1}$ **do**
- 20 $\mathcal{T}_{\mathbf{P}_X} \leftarrow \{t | \mathbf{P}_{x(t)} = \mathbf{P}_X\}$;
- 21 **foreach** t **in** $\mathcal{T}_{\mathbf{P}_X}$ **do**
- 22 **if** $\mathbf{P}_{x(t)} = \hat{\mathbf{P}}_{x(t)}|\mathcal{M}_Y$ **then**
- 23 Increment $PP_{v_x, v_y}[\mathbf{P}_X, \mathbf{P}_{y(t)}]$ by $\frac{1}{|\mathcal{T}_{\mathbf{P}_X}|}$
- 24 **return** PP_{v_x, v_y}

the estimated $\hat{\mathbf{P}}_{x(t)}|\mathcal{M}_Y$ corresponds to the actual $\mathbf{P}_{x(t)}$, that is

$$\mathbf{I}(\mathbf{P}_{x(t)}, \mathbf{P}_{y(t)}) = \begin{cases} 1, & \mathbf{P}_{x(t)} = \hat{\mathbf{P}}_{x(t)}|\mathcal{M}_Y \text{ and } \mathbf{P}_{y(t)} = \mathbf{P}_Y \\ 0, & \text{otherwise.} \end{cases} \quad (7)$$

Here, $\mathcal{T}_{\mathbf{P}_X} = \{t | \mathbf{P}_{x(t)} = \mathbf{P}_X\}$ represents the set of all time indices of points with pattern $\mathbf{P}_X$ on $\mathcal{M}_X$. By completing the normalized pattern prototype matrix PP_{v_i, v_j} for each directed stock pair, we quantify the matrix-valued causality for further interdependence analysis among stocks. Algorithm 1 outlines the overall workflow of pattern prototype recognition. It takes as input two given stocks and some hyperparameters, and works in four steps as illustrated.

As time steps gradually increase, new observations can continuously be incorporated into the sequence and participate in updating the pattern prototypes, while patterns from long ago can be gradually forgotten. In essence, we maintain a rolling observation window with a constant length T_o to adapt to the ever-volatile stock market.

B. Dynamic Network Inference

Essentially, pattern prototypes serve as intermediaries for inferring causal structures. By sidestepping the direct modeling of stock networks, it becomes feasible to integrate dynamics into causal inference. Since we perform a forward shift on the time

series of the receiver when identifying the causal effects from the driver in the pattern prototype recognition, here we infer in real time how the current state of the source stock v_x influences the target stock v_y at the next time step. These influences can be categorized into three types: positive (bullish), neutral, and negative (bearish) inducements, which are associated with symbols “ $\nearrow$ ”, “ $\rightarrow$ ”, and “ $\searrow$ ”, respectively. We define $\mathcal{P} = \{\nearrow, \rightarrow, \searrow\}$ as the pattern symbol set.

At current time step $t = L$, the historical movement pattern with length $E - 1$ of v_x can be extracted as

$$\tilde{\mathbf{P}}_{x(L)} = \text{signature}_{\theta}(\mathbf{r}_{x(L)}) \quad (8)$$

where $\mathbf{r}_{x(L)}$ is the temporal return pattern of $x(t)$ at time step L . However, as the state of v_y at time $t + 1$, $\mathbf{Y}^{L+1}$, is not observable, the historical movement pattern with only length $E - 2$ of v_y can be similarly extracted as

$$\tilde{\mathbf{P}}_{y(L)} = \text{signature}_{\theta}(\tilde{\mathbf{r}}_{y(L)}). \quad (9)$$

Here, the return pattern without the observation of $\mathbf{Y}^{L+1}$ is defined as

$$\tilde{\mathbf{r}}_{y(L)} = \left[\frac{y_L^{E-1} - y_L^E}{y_L^E}, \dots, \frac{y_L^2 - y_L^3}{y_L^3} \right] \quad (10)$$

where y_L^i represents the i th element on lagged-coordinate vectors $y(L)$. Note that $\tilde{\mathbf{r}}_{y(L)} \in \mathbb{R}^{E-2}$ differs from $\mathbf{r}_{y(t)} \in \mathbb{R}^{E-1}$ in (2) where t can only go up to $L - 1$.

Now, based on the two movement patterns, we can dynamically infer the directed causal link from v_x to v_y as a vector-valued edge

$$\mathbf{e}_{v_x, v_y}^L = [\alpha_{v_x, v_y}^{\nearrow}, \alpha_{v_x, v_y}^{\rightarrow}, \alpha_{v_x, v_y}^{\searrow}] \quad (11)$$

whose element for each $p \in \mathcal{P}$ is the attribute generated from the recognized pattern prototype matrix, denoted as

$$\alpha_{v_x, v_y}^p = PP_{v_x, v_y}[\tilde{\mathbf{P}}_{x(L)}, [\tilde{\mathbf{P}}_{y(L)} \| p]] \quad (12)$$

where “ $\|$ ” denotes vector concatenation. Such a retrieval process from pattern prototypes unveils both dynamics and causality in the stock networks, i.e., *state-dependent causality*. By leveraging this edge encoding method, each attribute α_{v_x, v_y}^p can be interpreted as the neighbor-induced bullish, neutral, or bearish signal influencing the next time step. Moreover, the magnitude of $\mathbf{e}_{v_x, v_y}^L$, $|\mathbf{e}_{v_x, v_y}^L|$, can be treated as the overall causal strength inferred from real-time movement patterns.

After inferring all mutual causal effects among stocks, we can dynamically construct a graph (or network) $\mathcal{G}^L = (\mathcal{V}, \mathbf{E}^L)$ at time $t = L$. Here, the node set $\mathcal{V}$ represents stocks, while the adjacency tensor $\mathbf{E}^L \in \mathbb{R}^{|\mathcal{V}| \times |\mathcal{V}| \times 3}$ encodes heterogeneous neighbor-induced movements, i.e., $\mathbf{E}_{v_x, v_y}^L = \mathbf{e}_{v_x, v_y}^L$ characterizes the tri-component causal effect. The step-by-step procedure for constructing the dynamic graph is summarized in Algorithm 2.

C. Causal Effect Aggregation

To integrate the causal effect between paired stocks and filter weak causality in the inferred stock network, we develop an

Algorithm 2: Dynamic Network Inference.

Input : The node set of all stocks $\mathcal{V}$; Closing price series of all stocks $\{\mathbf{X}, \mathbf{Y}, \dots\}$ with length L ; Pattern prototype matrices $\{PP_{v_x, v_y} | v_x, v_y \in \mathcal{V}\}$ of all stock pairs; Sliding time window size E ; Time leg τ ; Threshold θ .

Output : Dynamic Graph $\mathcal{G}^L = (\mathcal{V}, \mathbf{E}^L)$ at time $t = L$.

- 1 Initialize $\mathbf{E}^L$ as zero tensor of size $|\mathcal{V}| \times |\mathcal{V}| \times 3$;
- 2 **foreach** v_x **in** $\mathcal{V}$ **do**
- 3 **Step 1: Symbolize Current Movement Patterns.**
- 4 $x(L) \leftarrow [\mathbf{X}^L, \mathbf{X}^{L-\tau}, \dots, \mathbf{X}^{L-(E-1)\tau}]$;
- 5 Compute the return series of $x(L)$ as $\mathbf{r}_{x(L)}$;
- 6 Symbolize $\mathbf{r}_{x(L)}$ by threshold θ as $\tilde{\mathbf{P}}_{x(L)}$;
- 7 **foreach** v_y **in** $\mathcal{V}$ **and** $v_y \neq v_x$ **do**
- 8 $y(L) \leftarrow [\mathbf{Y}^{L+1-\tau}, \mathbf{Y}^{L+1-2\tau}, \dots, \mathbf{Y}^{L+1-(E-1)\tau}]$;
- 9 Compute the reduced return series of $y(L)$ as $\tilde{\mathbf{r}}_{y(L)}$;
- 10 Symbolize $\tilde{\mathbf{r}}_{y(L)}$ by threshold θ as $\tilde{\mathbf{P}}_{y(L)}$;
- 11 **Step 2: Retrieve Causal Effects as Edges.**
- 12 **foreach** p **in** $\{\nearrow, \rightarrow, \searrow\}$ **do**
- 13 $\alpha_{v_x, v_y}^p \leftarrow PP_{v_x, v_y}[\tilde{\mathbf{P}}_{x(L)}, [\tilde{\mathbf{P}}_{y(L)} \| p]]$;
- 14 $\mathbf{E}_{v_x, v_y}^L \leftarrow [\alpha_{v_x, v_y}^{\nearrow}, \alpha_{v_x, v_y}^{\rightarrow}, \alpha_{v_x, v_y}^{\searrow}]$;
- 15 **return** $\mathcal{G}^L = (\mathcal{V}, \mathbf{E}^L)$

aggregation mechanism with multiple layers for modeling the propagation of heterogeneous interventions.

Stocks are not only responsive to their historical prices but also susceptible to momentum spillover effects from related stocks, often attributed to market anomalies [4]. Typically, technical indicators of each stock serve as a feature vector on the trading day [7], [8]. Time series modeling aims to capture temporal dependencies within a sequence of continuous price points or indicators. Recent studies have suggested that RNNs may outperform Transformers [58] in time-series tasks [59]. Building upon previous work [8], [10], our approach employs a single bidirectional LSTM (Bi-LSTM) [25] module with two d_n -dimensional hidden layers to extract price information from the past T_f trading days. For stock $v_y \in \mathcal{V}$, this module fuses the historical signals to generate the initial node embedding $\mathbf{n}_{v_y}^{L, (0)} \in \mathbb{R}^{d_n}$ at time step $t = L$

$$\mathbf{n}_{v_y}^{L, (0)} = \text{Bi-LSTM}(\mathbf{F}_{v_y}^{(L-T_f:L)}) \quad (13)$$

where $\mathbf{F}_{v_y}^{(L-T_f:L)} = [\mathbf{f}_{v_y}^{L-(T_f-1)}, \dots, \mathbf{f}_{v_y}^L]$ denotes the matrix concatenating historical signals and $\mathbf{f}_{v_y}^L \in \mathbb{R}^{d_f}$ is the technical indicators of stock v_y . By performing time-series embedding outlined in (13), the learning module captures the response representation of the target stock from a multi-sequence excitation signal.

As a next step, we encode the layer-wise causal effect from v_x to v_y as a d_u -dimensional hidden vector through an aggregator

$$\mathbf{u}_{v_x, v_y}^{L, (k)} = \sigma \left(\mathbf{W}_{un}^{(k)} [\mathbf{n}_{v_x}^{L, (k-1)} \| \mathbf{e}_{v_x, v_y}^L] + \mathbf{b}_u^{(k)} \right) \quad (14)$$

where $\mathbf{b}_u^{(k)} \in \mathbb{R}^{d_u}$ and $\mathbf{W}_{un}^{(k)}$ are learnable parameters for the k th layer and $\sigma(\cdot)$ represents a nonlinear activation function. Note that when $k = 1$, $\mathbf{W}_{un}^{(k)} \in \mathbb{R}^{d_u \times (d_n+3)}$, and when $k \geq 2$, $\mathbf{W}_{un}^{(k)} \in \mathbb{R}^{d_u \times (d_u+3)}$. Since not all pairs of stocks exhibit significant causal interactions, we employ a gating mechanism to filter

out weak causality, and then the node representations can be updated by aggregating the causal effect from the neighborhood to v_y , that is

$$\mathbf{n}_{v_y}^{L,(k)} = \sum_{v_x \in \mathcal{V}} \frac{\exp(\beta_{v_x, v_y}^L) \mathbf{u}_{v_x, v_y}^{L,(k)}}{\sum_{v_i \in \mathcal{V}, \beta_{v_i, v_y}^L \neq 0} \exp(\beta_{v_i, v_y}^L)} \quad (15)$$

where

$$\beta_{v_x, v_y}^L = \begin{cases} |\mathbf{e}_{v_x, v_y}^L|, & |\mathbf{e}_{v_x, v_y}^L| \geq \lambda \\ 0, & \text{otherwise.} \end{cases} \quad (16)$$

Here, the causal strength is quantified by the magnitude of $\mathbf{e}_{v_x, v_y}^L$, and we normalize the non-zero causal strength with a softmax function. λ is a hyperparameter to control the filtering threshold.

After K rounds of aggregation, this module learns and embeds solid causal interactions among stocks to accurately model the node-level momentum spillover effects, rather than only capturing superficial correlation.

D. Prediction and Optimization

Here we concatenate all the node representations $\{\mathbf{n}_{v_i}^{L,(k)} | k = 0, \dots, K\}$ and feed them into a multi-layer perceptron (MLP) to generate the predicted movement $\hat{\mathbf{m}}_{v_i}^{L+1} \in \mathbb{R}^2$ of stock $v_i \in \mathcal{V}$ for the next trading day $L + 1$

$$\hat{\mathbf{m}}_{v_i}^{L+1} = [\hat{m}_{v_i}^-, \hat{m}_{v_i}^+] = \text{softmax} \left(\text{MLP}_{v_i} \left(\left\| \mathbf{n}_{v_i}^{L,(k)} \right\|_{k=0}^K \right) \right) \quad (17)$$

where $\hat{m}_{v_i}^-$ and $\hat{m}_{v_i}^+$ represent the movement probability for fall and rise, respectively, and the learnable parameters of MLP_{v_i} are tailored for individual stocks. The initial node representation $\mathbf{n}_{v_i}^{L,(0)}$ is the learned intrinsic time-series features of stock v_i , serving as a response to its historical price signals, while other node representations integrate causal interactions from multi-order neighbors.

Finally, we use the cross-entropy loss to directly encourage the output to capture the price movements of all stocks

$$\mathcal{L} = -\frac{1}{|\mathcal{V}|} \sum_{v_i \in \mathcal{V}} ((1 - m_{v_i}^{L+1}) \log(\hat{m}_{v_i}^-) + m_{v_i}^{L+1} \log(\hat{m}_{v_i}^+)) \quad (18)$$

where $m_{v_i}^{L+1}$ denotes the ground-truth movement of stock v_i at time $L + 1$. By minimizing $\mathcal{L}$ across all time steps, our model learns all parameters to enhance stock movement prediction performance.

V. EXPERIMENTS

In this section, we conduct a series of experiments to comprehensively evaluate the complexity, effectiveness, robustness, and interpretability of the proposed DC-GNN.

A. Evaluation Setup

1) *Datasets*: We collected historical trading data for NASDAQ 100 entities and S&P 500 entities spanning from Jan.

2014 to Dec. 2019, sourced from Yahoo Finance⁴ and Nasdaq Data Link.⁵ A total of 118 stocks from the NASDAQ 100 Index and 510 stocks from the S&P 500 Index are incorporated into our evaluation, all of which had complete transaction data throughout the entire period.

2) *Compared Baselines*: We conduct a comparative analysis of the proposed model against 12 baselines categorized into two groups: 1) Sequential models, including LSTM [25], Transformer [58], and Pathformer [60]; 2) state-of-the-art graph-based methods, including STHAN-SR [34], ESTIMATE [8], ECHO-GL [31], RT-GCN [29], SAMBA [61], THGNN [7], S-GRN [6], DANCMP [10], PTER [9], and MD-GNN [32]. For an equitable comparison within the same prediction task, we introduce minor adjustments to the objective functions of these methods to predict future stock movements.

3) *Evaluation Metrics*: We evaluate the performance of different models on the stock movement prediction task using two standard metrics: accuracy (ACC) and Matthew's correlation coefficient (MCC) [62]. In particular, MCC performs more precisely when data is skewed by accounting for the true negatives. Furthermore, we employ a simulated trading portfolio to assess the backtesting profitability of various models, considering three metrics: annualized return rate (ARR), maximal drawdown (MDD), and annualized sharp ratio (ASR). Further, ARR is calculated by summing all daily returns, offering a direct measure of annualized profitability averaged over all years. MDD is a risk indicator represented by the maximum loss of return from peak to trough. ASR evaluates returns relative to volatility, providing insights into the risk-adjusted profitability, defined as $\text{ASR} = \frac{\text{ARR} - R_f}{\sigma_p}$. Here, σ_p denotes the annualized standard deviation of the portfolio, serving as a measure of volatility, and R_f denotes the average return rate of a risk-free asset, typically represented by government bonds.

4) *Implementation Details*: Both datasets were split into two periods: Jan. 2014 to Jun. 2016 for recognizing the original pattern prototypes and Jul. 2016 to Dec. 2019 for training and testing. We adopted a rolling window fashion [8] in 12 sub-experiments for evaluation. Considering the high timeliness of financial data, we set the length of the test set to one month. Specifically, in each sub-experiment, the data covering 31 months was divided chronologically into three segments: 28 months for training, 2 months for validation, and 1 mo for testing. Note that during training and testing, the prototypes were only updated after the specific instance had been backpropagated to prevent any data overlap. For parameter settings, a grid search was employed to determine the optimal hyperparameters. For the NASDAQ 100 dataset and the S&P 500 dataset, the hyperparameters were respectively configured as follows: τ to $\{1, 1\}$, E to $\{3, 3\}$, T_o to $\{520, 460\}$, T_f to $\{12, 15\}$, d_n to $\{384, 512\}$, d_u to $\{128, 192\}$, K to $\{2, 2\}$, λ to $\{0.38, 0.46\}$. For each stock, the threshold θ was set to be half of the standard deviation of its historical returns. All learnable parameters were initialized by Glorot initialization [63] and trained for a maximum of 200 epochs using the AdamW optimizer, with a learning rate of $5e-4$.

⁴<https://finance.yahoo.com>

⁵<https://data.nasdaq.com>

TABLE I
AVERAGE EMPIRICAL RESULTS OF COMPLEXITY ANALYSIS

Task	Metric	NASDAQ 100	S&P 500
PPR	Overall runtime	2.72 min	48.97 min
	Resident set size	231.76 MB	271.05 MB
	Virtual memory size	842.21 MB	1035.70 MB
DC-GNN	Training time per epoch	2.06 min	23.68 min
	GPU memory usage	3.95 GB	4.73 GB

B. Complexity Analysis

To assess the computational efficiency of the proposed method, we report the theoretical complexity, empirical runtime, and memory measurements. All experiments were conducted on a Linux server equipped with an Intel Xeon E5-2640 v4 CPU, 256 GB RAM, and two NVIDIA TITAN V GPUs (12 GB HBM2 memory each).

On the one hand, the time complexity of pattern prototype recognition (PPR) across all stock pairs is dominated by cross-mapped pattern estimation and nearest-neighbor search, i.e., $O(N^2 \cdot L^2 \cdot E)$, where N is the number of stocks, L is the length of the closing price series, and E is the delay embedding dimension. Furthermore, the space complexity is primarily determined by the storage of prototype matrices, resulting in a complexity of $O(N^2 \cdot 3^{2(E-1)})$. Table I (top) summarizes the runtime and memory cost of running PPR across both datasets. Empirically, prototype recognition requires processing $N(N-1)$ stock pairs over L days, which is computationally intensive. However, memory usage is relatively stable and does not scale as dramatically as runtime, as the prototype matrices are sparsely populated in practice.

On the other hand, when optimizing DC-GNN, since a dynamic graph must be reconstructed on every trading day, the procedure is time-consuming despite being parallelized. Table I (bottom) summarizes the average training runtime and GPU memory usage for model optimization. The runtime gap (11.5 $\times$) reflects the cost of inferring dynamic graphs using all $N(N-1)$ pattern prototypes. In contrast, GPU memory usage increases only modestly, since the dominant cost comes from recurrent graph modeling rather than model parameterization.

These results suggest that DC-GNN is memory-efficient but computationally intensive for large-scale markets. While the prototype representation ensures that storage costs remain manageable, the daily reconstruction of dynamic graphs constitutes the main bottleneck. This highlights the importance of efficient nearest-neighbor search and parallelization strategies for scaling DC-GNN to even larger universes of assets.

C. Effectiveness Evaluation

1) *Stock Movement Prediction*: We report the results of all methods on stock movement prediction in Table II, where the metrics are computed across all stocks within each testing period and the means and standard deviations of them are calculated over 12 testing periods. Furthermore, we report the results of the Diebold-Mariano test [64] in Table III to examine whether there is a statistically significant difference between each baseline and

TABLE II
PERFORMANCE EVALUATION OF STOCK MOVEMENT PREDICTION (MEAN $\pm$ STANDARD DEVIATION)

Method	NASDAQ 100		S&P 500	
	ACC (%)	MCC	ACC (%)	MCC
LSTM	52.57 $\pm$ 0.39	0.051 $\pm$ 0.022	52.39 $\pm$ 0.52	0.049 $\pm$ 0.038
Transformer	52.09 $\pm$ 0.61	0.046 $\pm$ 0.035	51.76 $\pm$ 0.59	0.040 $\pm$ 0.029
Pathformer	53.62 $\pm$ 0.31	0.072 $\pm$ 0.018	52.12 $\pm$ 0.44	0.044 $\pm$ 0.026
STHAN-SR	54.76 $\pm$ 0.83	0.091 $\pm$ 0.051	53.23 $\pm$ 0.44	0.073 $\pm$ 0.031
ESTIMATE	54.17 $\pm$ 0.67	0.082 $\pm$ 0.056	53.54 $\pm$ 0.59	0.071 $\pm$ 0.035
ECHO-GL	55.07 $\pm$ 0.65	0.094 $\pm$ 0.069	55.21 $\pm$ 0.56	0.088 $\pm$ 0.059
RT-GCN	55.68 $\pm$ 0.79	0.113 $\pm$ 0.057	54.49 $\pm$ 0.78	0.074 $\pm$ 0.049
SAMBA	56.46 $\pm$ 1.12	0.127 $\pm$ 0.045	54.89 $\pm$ 0.91	0.098 $\pm$ 0.053
THGNN	56.51 $\pm$ 0.96	0.124 $\pm$ 0.076	55.91 $\pm$ 0.67	0.114 $\pm$ 0.061
S-GRN	56.85 $\pm$ 0.80	0.132 $\pm$ 0.044	55.73 $\pm$ 0.43	0.121 $\pm$ 0.036
DANSMP	57.85 $\pm$ 0.72	0.149 $\pm$ 0.076	56.27 $\pm$ 1.17	0.128 $\pm$ 0.094
PTER	57.64 $\pm$ 0.51	0.155 $\pm$ 0.033	56.83 $\pm$ 0.82	0.147 $\pm$ 0.060
MD-GNN	58.33 $\pm$ 1.21	0.168 $\pm$ 0.089	57.39 $\pm$ 1.40	0.152 $\pm$ 0.089
DC-GNN	59.82$\pm$0.84	0.196$\pm$0.061	58.26$\pm$0.79	0.170$\pm$0.065

TABLE III
DIEBOLD-MARIANO TEST RESULTS BETWEEN DC-GNN AND EACH BASELINE MODEL

Baseline	NASDAQ 100	S&P 500
LSTM	0.0000***	0.0000***
Transformer	0.0000***	0.0000***
Pathformer	0.0000***	0.0000***
STHAN-SR	0.0000***	0.0006***
ESTIMATE	0.0081***	0.0351**
ECHO-GL	0.0057***	0.0207**
RT-GCN	0.0033***	0.0322**
SAMBA	0.0242**	0.0397**
THGNN	0.0485**	0.0318**
S-GRN	0.0290**	0.0044***
DANSMP	0.0154**	0.0495**
PTER	0.0404**	0.0370**
MD-GNN	0.0466**	0.0861*

Note: *, **, and *** represent statistical significance at the levels of 10%, 5%, and 1%, respectively.

our DC-GNN in the predicted accuracy of rise movement across all test days.

Main results: The results in Table II indicate that our DC-GNN outperforms all state-of-the-art baselines on both datasets in terms of ACC and MCC. As shown in Table III, our model demonstrates superiority over all baselines at a significance level of at least 10%. For each baseline, our model exhibits significantly better predictive accuracy on at least one dataset at a significance level of 5%. This confirms the effectiveness of inferring a dynamic and causal network from the identified pattern prototypes for modeling stock interactions. Our model not only discerns the directed causality among stocks but also circumvents the impact of diverse price levels across different stocks through symbolic dynamics modeling. By dynamically adapting to market backgrounds, DC-GNN can construct robust real-time topology structures, showcasing superior prediction performance.

Analysis: a) LSTM, Transformer, and Pathformer, limited by their capacity to grasp the intricate interdependencies among

different entities, exhibit worse performance when compared to graph-based methods. This underscores that relying solely on price signals or indicators is insufficient to support accurate movement prediction. b) The overall performance for various models, excluding sequential models, tends to be subpar on the S&P 500 dataset. This can be ascribed to the increased complexity of stock networks and the inevitable aggregation of more noise. However, DC-GNN showcases the best performance even on the S&P 500 dataset, indicating that state-dependent causality is more effective than static correlation. c) STHAN-SR, ESTIMATE, ECHO-GL, RT-GCN, and SAMBA, accounting for stock relations or even heterogeneous relations, outperform classical sequential models. However, they demonstrate inferior performance over other graph-based methods due to their limited consideration of time-varying connections. d) The stock networks employed by THGNN, S-GRN, and MD-GNN are structurally dynamic, which contributes to a slight improvement in their prediction performance. However, they are surpassed by DC-GNN as they depend on correlation-based edges. e) Both DANSMIP and PTER construct complex knowledge graphs or hypernetworks containing inter- and intra-sector relations or micro-object and macro-scenario knowledge. These methods result in a notable enhancement. Nevertheless, besides these two methods, all graph-based models have ignored the causal interventions arising from daily trading signals. They show inferior performance compared with DC-GNN in terms of ACC and MCC eventually.

2) *Backtesting Profitability*: To intuitively reveal the profitability and risk control performance of different models in the real trading environment, we implemented a trading strategy to simulate real-world investment on historical data, a special type of cross-validation called backtesting. In the execution, stocks within the designated stock pool were ranked daily based on their movement prediction scores. We purchased a maximum of 5 of the highest-ranked stocks from the top 15 stocks, provided they were not already in the portfolio. Simultaneously, an equivalent quantity of the lowest-ranked stocks was sold. This strategy effectively controlled the turnover rate, maintaining stable portfolio holdings with a total of 15 stocks. We initialized the account capital at \$10 million and factored in a transaction cost of 0.05% when buying shares, as well as 0.15% when selling shares. The average results of all models are presented in Table IV.

Main results: The findings illustrate that DC-GNN achieves optimal performance by balancing profitability and risk mitigation when backtesting. The highest ARR score underscores its exceptional profitability, and the highest ASR signifies its remarkable overall performance in risk-adjusted returns. Although S-GRN shows slightly better MDD performance on the NASDAQ 100 dataset, DC-GNN demonstrates superior overall performance through more outstanding profitability. This can be attributed to our method's proficiency in inferring robust causal relationships in real time, which directly models the underlying mechanisms of stock interactions for sensitively identifying the dominant inducement on specific stock movement. Moreover, DC-GNN can continuously adapt to market conditions, uncovering weak risks that are challenging to detect through hard-coded correlation-based relationships.

TABLE IV
PERFORMANCE EVALUATION OF BACKTESTING PROFITABILITY

Method	NASDAQ 100			S&P 500		
	ARR	MDD	ASR	ARR	MDD	ASR
LSTM	15.15%	-8.248%	1.406	13.73%	-7.764%	1.151
Transformer	13.93%	-9.106%	1.091	12.28%	-8.254%	0.998
Pathformer	16.61%	-9.416%	1.462	13.20%	-7.981%	1.135
STHAN-SR	18.34%	-8.545%	1.509	14.64%	-9.613%	1.351
ESTIMATE	17.52%	-9.907%	1.487	14.45%	-10.49%	1.122
ECHO-GL	19.66%	-9.622%	1.625	15.92%	-9.764%	1.382
RT-GCN	19.43%	-8.838%	1.702	14.36%	-8.547%	1.412
SAMBA	19.96%	-8.281%	1.640	15.29%	-9.005%	1.356
THGNN	20.63%	-8.724%	1.769	15.47%	-8.463%	1.508
S-GRN	22.49%	-6.209%	2.028	18.95%	-6.936%	1.611
DANSMIP	24.26%	-10.35%	1.686	20.56%	-9.264%	1.643
PTER	26.18%	-7.329%	2.251	22.78%	-8.733%	1.826
MD-GNN	27.14%	-6.825%	2.304	24.38%	-7.440%	1.967
DC-GNN	29.54%	-6.418%	2.470	26.62%	-6.821%	2.189

Analysis: a) Combining the findings with Table II, we observe that, as a general rule, the more accurate a model's prediction regarding movement, the greater its potential for earning. This aligns well with investment experience. b) LSTM, Transformer, and Pathformer treat each stock as an isolated node, which partially mitigates some external risks, but hinders them from modeling momentum spillover, ultimately limiting their profitability. c) STHAN-SR, ESTIMATE, ECHO-GL, RT-GCN, and SAMBA cannot capture the inherent dynamics of the network, thus missing out on timely risk feedback and lacking the ability to make real-time risk adjustments. As a result, their MDD performance lags behind even that of sequential models. d) S-GRN is tailor-made for capturing variant risk propagation processes among entities represented as time series, effectively controlling risks with the lowest MDD scores, but at the expense of a significant portion of profit potential. e) THGNN, DANSMIP, PTER, and MD-GNN overlook the causality among stocks. These shortcomings result in their inferior performance when compared to DC-GNN. Furthermore, DC-GNN infers a dynamic network at each time step, facilitating a balance between returns and risks.

3) *Ablation Study*: To systematically verify the effectiveness of each component in our model, we design 5 variants of DC-GNN for an ablation study: a) *w/o causality*, which computes the heterogeneous edge weights using partial correlation coefficients of different indicators rather than causality. b) *w/o dynamics*, which directly assigns weight to an edge based on three static heterogeneous relationships, represented respectively by the mean of positive, negative, and neutral causality in each original prototype matrix. c) *w/o time shifting*, which aligns the elements of $x(t)$ and $y(t)$ for synchrony, without imposing a fixed time delay between each element in $x(t)$ and its corresponding element in $y(t)$. d) $-PPR + CCM$, which replaces the pattern prototype recognition module with CCM [21] to dynamically infer directed homogeneous causal effects by sliding the temporal window. e) *w/o heterogeneity*, which defines the homogeneous edge of each directed stock pair based on the causal strength $|e_{v_x, v_y}^L|$.

The performance of accuracy shown in Fig. 3 underscores the significant role played by each component within DC-GNN.

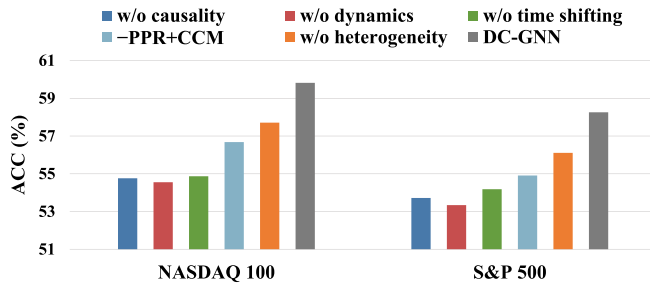


Fig. 3. Accuracy of different DC-GNN variants.

The inclusion or exclusion of specific components profoundly influences DC-GNN's predictive capability. More specifically, the core of DC-GNN lies in the dynamics (*w/o dynamics*) and pattern-specific causality (*w/o causality*), which are pivotal in bolstering accuracy. Furthermore, the one time step temporal shift between $x(t)$ and $y(t)$ enables the pattern prototype to directly capture the forward causal effects of time series on a temporal scale (*w/o time shifting*). As stated in Section III-C, while CCM ($-PPR + CCM$) captures causal relationships to some extent, the non-stationarity of trading signals poses a challenge for nearest neighbors on the shadow manifold to shrink sufficiently until causal strength converges. Thus it proves inadequate to address complex financial scenarios. Finally, through heterogeneous modeling of positive, negative, and neutral inducements, DC-GNN captures peer impacts beyond the reach of homogeneous edges (*w/o heterogeneity*), further enhancing prediction performance.

D. Robustness Evaluation

1) *Hyperparameter Sensitivity Analysis*: We explore the impacts of different hyperparameter configurations on DC-GNN in Fig. 4 to analyze its sensitivity and robustness, including the filtering threshold λ , symbolization threshold θ (where σ denotes the historical return standard deviation for each stock), delay embedding dimension E , time lag τ , two lookback window sizes T_o and T_f , and two representation dimensions d_n and d_u . The chosen hyperparameter ranges are based on prior financial time-series studies [8], [35] and domain knowledge. Specifically, $T_f \in [6, 27]$ covers approximately one trading week to one trading month to capture short-term market dynamics. $T_o \in [420, 560]$ spans around two years of trading days, modeling long-term causal dependencies in pattern prototypes while avoiding the incorporation of stale or outdated information. Representation dimensions $d_n \in [256, 640]$ and $d_u \in [64, 448]$ are set to ensure sufficient expressive power without overfitting. Since the node representations are learned from multiple technical indicators, they require a higher dimensionality (d_n) than the edge representations (d_u) to adequately encode the node-level information. These choices are further validated through sensitivity analysis.

To capture latent causal relations under more complex market scenarios, we increase the embedding dimension E at the top of the second column of Fig. 4. However, accuracy improves but declines thereafter, as larger E leads to sparsity of prototype

matrices ($3^{E-1} \times 3^{E-1}$). Such sparsity makes it more difficult for the model to reliably estimate causal relations, as many pattern combinations occur rarely or not at all in the data. One possible remedy is to enlarge the observation window T_o so that more temporal patterns are observed to alleviate data sparsity. However, due to market evolution and frequent policy or structural changes, overly large T_o inevitably introduces outdated or irrelevant patterns, ultimately degrading prototype quality. This reveals a trade-off: while larger E helps capture long-term dependencies, it also exacerbates sparsity and vulnerability to stale information, requiring careful calibration of both E and T_o .

As for other hyperparameters, the results indicate that selecting an appropriate value of λ can maintain significant causality while minimizing the introduction of additional noise. For the symbolization threshold θ , a smaller value leads to excessive sensitivity, where noisy fluctuations in low-volatility stocks are misinterpreted as signals, while a larger value suppresses weaker yet meaningful short-term signals, particularly in high-volatility stocks. Increasing the window size T_f enhances the capacity to capture long-term trends but overlooks the market dynamics. Furthermore, a large value of τ poses challenges for DC-GNN in capturing fine-grained features effectively, inevitably diminishing prediction performance. Finally, excessively low and high representation dimensions (d_n and d_u) may reduce prediction quality due to underfitting and overfitting, respectively. Overall, DC-GNN demonstrates stability around the optimal setting, suggesting its relative robustness across various hyperparameters.

2) *Stress Test During Market Crash*: We also conducted a stress test during the market crash period from Oct. 2018 to Dec. 2018 to evaluate our model's robustness amidst turbulent market conditions. Notably, the temporal length of the training set had to be compressed to 25 months in this scenario.

Benchmarking against the market: Fig. 5 shows the cumulative returns of the NASDAQ 100 Index and our method. Due to the Federal Reserve's persistent interest rate hikes in the first half of 2018, the stock market came under pressure, ultimately experiencing a widespread and highly volatile downturn. This is exemplified by the steep declines from Oct. 2018 for the NASDAQ 100 Index. As the index plunged into successive downturns, DC-GNN struggled to navigate the prevailing panic sentiment and occasionally made erroneous judgments. However, as it gradually adapted to the evolving market conditions within the time window, DC-GNN displayed an aptitude for risk management to a certain degree, avoiding more significant economic losses. These findings indicate that our model exhibits resilience in extreme market conditions but falls short of generating profits. Its negative impact is limited because striking a balance between risk control and profit generation is more crucial than focusing solely on profit.

Stability of the symbolization threshold: In addition to the market crash analysis, we stress-tested the effect of the symbolization threshold θ in the crisis period. Fig. 6 compares the cumulative returns of DC-GNN with different θ values during the same crash period. The results show that a smaller threshold (0.25σ) exhibits high sensitivity during the early stages of market crashes (e.g., in October). This sensitivity enables it

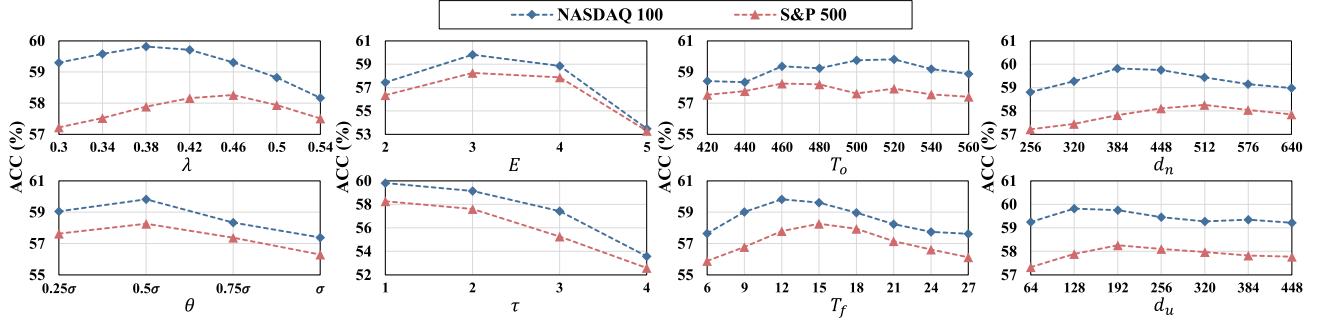


Fig. 4. Hyperparameter sensitivity analysis of DC-GNN on two datasets.

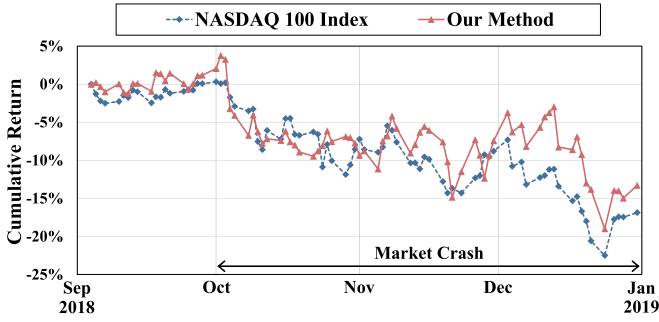
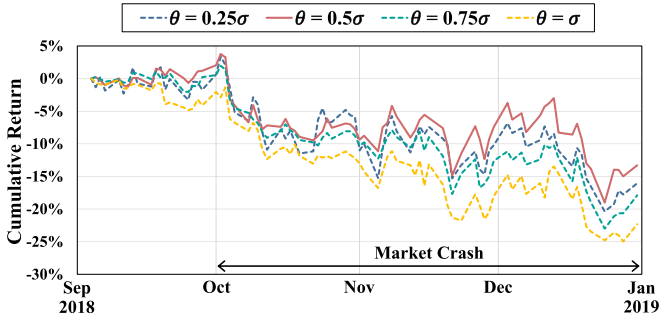


Fig. 5. The profitability of our method against the NASDAQ 100 Index during the market crash period.


 Fig. 6. The impact of symbolization threshold θ in the NASDAQ 100 dataset during the market crash period, where σ denotes the historical return standard deviation for each stock.

to capture subtle signals in highly volatile environments and thereby avoid substantial losses. However, such a radical behavior also increases susceptibility to noise, which compromises stability and leads to underperformance (against 0.5σ) in subsequent periods. By contrast, when θ is larger (0.75σ and σ), the benefit from suppressing noise is outweighed by the loss from overlooking meaningful signals, resulting in diminished profitability and reduced resilience to risk. This highlights the trade-off between noise suppression and signal capture. Overall, the experiment demonstrates that DC-GNN retains a certain degree of robustness to threshold variations under volatile market conditions. Nevertheless, the observed decline in profitability and risk resilience highlights the necessity of cautious parameter selection.

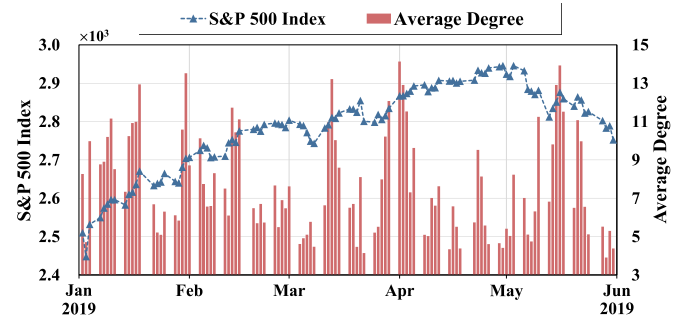


Fig. 7. Case study on the structural dynamics.

E. Interpretability Evaluation

To visualize how DC-GNN captures structural dynamics, we conducted a case study on the S&P 500 test sets to explore its interpretability. We extracted a subgraph from the inferred stock networks, focusing on the positive attribute (α_{v_x, v_y}^+) with significant weight, to observe how the average degree of this homogeneous graph is influenced by overall market volatility. As illustrated in Fig. 7, during each bullish period characterized by an upward trajectory in the S&P 500 Index, the number of neighbor-induced positive effects within the inferred stock network increases distinctly. Conversely, in bearish or neutral market phases, this trend subsides. This observation underscores that DC-GNN dynamically assigns weights to neighbors with significant positive impacts, facilitating effective causal interactions between nodes. By incorporating the dynamics of stock networks, our model showcases remarkable adaptability to diverse market conditions, hinting at its vast potential across various applications.

VI. CONCLUSION

In this paper, we propose the DC-GNN for stock movement prediction. This framework aims to infer *state-dependent causality* that is manifested in that only when a stock shows a specific pattern of movements will it be triggered and exert a causal effect on related stocks. By leveraging symbolic dynamics, our method overcomes the non-stationary price levels of diverse stocks and identifies neighbor-induced positive, negative, and neutral effects to quantify heterogeneous

interventions. Extensive experiments conducted on two real-world trading datasets underscore the effectiveness, robustness, and interpretability of DC-GNN in modeling both dynamics and causality. Beyond finance, our methodology holds the potential to be extended into other multivariate time-series analysis and dynamic graph modeling domains, such as topology tracking for power systems [65] and latent social network inference [66], whose full power has yet to be fully harnessed. The future extension of our work involves incorporating different financial instruments denoted as heterogeneous nodes to explore more comprehensive and informative dynamic-causal networks.

REFERENCES

- [1] W. Chen and Y. Wang, "Dhmo: Diffusion generated hierarchical multi-granular expertise for stock prediction," in *Proc. AAAI Conf. Artif. Intell.*, 2025, pp. 11490–11499.
- [2] S. Li, W. Liao, Y. Chen, and R. Yan, "PEN: Prediction-explanation network to forecast stock price movement with better explainability," in *Proc. AAAI Conf. Artif. Intell.*, 2023, pp. 5187–5194.
- [3] K. Seo and J. Yang, "Exploring multi-time context vector and randomness for stock movement prediction," in *Proc. IEEE Int. Conf. Big Data*, 2022, pp. 1114–1123.
- [4] R. Cheng and Q. Li, "Modeling the momentum spillover effect for stock prediction via attribute-driven graph attention networks," in *Proc. AAAI Conf. Artif. Intell.*, 2021, pp. 55–62.
- [5] L. Huang, Y. Xie, Q. Gao, K. Zhang, G. Liu, and X. Chen, "Progressive dependency representation learning for stock ranking in uncertain risk contrasting," in *Proc. 31st ACM SIGKDD Conf. Knowl. Discov. Data Mining V*, 2025, pp. 532–543.
- [6] R. Cheng and Q. Li, "Subsequence-based graph routing network for capturing multiple risk propagation processes," in *Proc. Int. Joint Conf. Artif. Intell.*, 2022, pp. 3810–3816.
- [7] S. Xiang, D. Cheng, C. Shang, Y. Zhang, and Y. Liang, "Temporal and heterogeneous graph neural network for financial time series prediction," in *Proc. 31st ACM Int. Conf. Inf. Knowl. Manage.*, 2022, pp. 3584–3593.
- [8] T. T. Huynh et al., "Efficient integration of multi-order dynamics and internal dynamics in stock movement prediction," in *Proc. 16th ACM Int. Conf. Web Search Data Mining*, 2023, pp. 850–858.
- [9] H. Wang, T. Wang, S. Li, J. Zheng, W. Chen, and W. Chen, "Agree to disagree: Personalized temporal embedding and routing for stock forecast," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 9, pp. 4398–4410, Sep. 2024.
- [10] Y. Zhao et al., "Stock movement prediction based on bi-typed hybrid-relational market knowledge graph via dual attention networks," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 8, pp. 8559–8571, Aug. 2023.
- [11] F. Feng, X. He, X. Wang, C. Luo, Y. Liu, and T.-S. Chua, "Temporal relational ranking for stock prediction," *ACM Trans. Inf. Syst.*, vol. 37, no. 2, pp. 1–30, 2019.
- [12] L. Yao et al., "A survey on causal inference," *ACM Trans. Knowl. Discov. Data (TKDD)*, vol. 15, no. 5, pp. 1–46, 2021.
- [13] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *Proc. Int. Conf. Learn. Representations*, 2018, pp. 1–12.
- [14] S. Brody, U. Alon, and E. Yahav, "How attentive are graph attention networks," in *Proc. Int. Conf. Learn. Representations*, 2022, pp. 1–26.
- [15] H. Liu, B. Hu, Y. Zhou, and Y. Zhou, "Responding to news sensitively in stock attention networks via prompt-adaptive trimodal model," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 36, no. 6, pp. 10099–10113, Jun. 2025.
- [16] Y. Wang, Y. Sun, Z. Liu, S. E. Sarma, M. M. Bronstein, and J. M. Solomon, "Dynamic graph CNN for learning on point clouds," *ACM Trans. Graph.*, vol. 38, no. 5, pp. 1–12, 2019.
- [17] J. Halcrow, A. Mosoi, S. Ruth, and B. Perozzi, "Grale: Designing networks for graph learning," in *Proc. 26th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2020, pp. 2523–2532.
- [18] C. W. Granger, "Investigating causal relations by econometric models and cross-spectral methods," *Econometrica: J. Econometric Soc.*, vol. 37, pp. 424–438, 1969.
- [19] R. F. Engle and C. W. Granger, "Co-integration and error correction: Representation, estimation, and testing," *Econometrica: J. Econometric Soc.*, pp. 251–276, 1987.
- [20] T. Schreiber, "Measuring information transfer," *Phys. Rev. Lett.*, vol. 85, no. 2, 2000, Art. no. 461.
- [21] G. Sugihara et al., "Detecting causality in complex ecosystems," *Science*, vol. 338, no. 6106, pp. 496–500, 2012.
- [22] S. K. Stavroglou, A. A. Pantelous, H. E. Stanley, and K. M. Zuev, "Hidden interactions in financial markets," *Proc. Nat. Acad. Sci.*, vol. 116, no. 22, pp. 10646–10651, 2019.
- [23] S. K. Stavroglou, A. A. Pantelous, H. E. Stanley, and K. M. Zuev, "Unveiling causal interactions in complex systems," *Proc. Nat. Acad. Sci.*, vol. 117, no. 14, pp. 7599–7605, 2020.
- [24] J. Pearl, *Causality*. Cambridge, U.K.: Cambridge Univ. Press, 2009.
- [25] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [26] H. Wang, T. Wang, and Y. Li, "Incorporating expert-based investment opinion signals in stock prediction: A deep learning framework," in *Proc. AAAI Conf. Artif. Intell.*, 2020, pp. 971–978.
- [27] S. Lin et al., "Cspo: Cross-market synergistic stock price movement forecasting with pseudo-volatility optimization," 2025, *arXiv:2503.22740*.
- [28] Y. Liu et al., "Digger-guider: High-frequency factor extraction for stock trend prediction," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 12, pp. 7973–7985, Dec. 2024.
- [29] Z. Zheng, J. Shao, J. Zhu, and H. T. Shen, "Relational temporal graph convolutional networks for ranking-based stock prediction," in *Proc. IEEE 39th Int. Conf. Data Eng.*, 2023, pp. 123–136.
- [30] Q. Gao, Z. Liu, L. Huang, K. Zhang, J. Wang, and G. Liu, "Relational stock selection via probabilistic state space learning," *IEEE Trans. Knowl. Data Eng.*, vol. 37, no. 2, pp. 865–880, Feb. 2025.
- [31] M. Liu, M. Zhu, X. Wang, G. Ma, J. Yin, and X. Zheng, "Echo-GL: Earnings calls-driven heterogeneous graph learning for stock movement prediction," in *Proc. AAAI Conf. Artif. Intell.*, 2024, pp. 13972–13980.
- [32] H. Qian et al., "MDGNN: Multi-relational dynamic graph neural network for comprehensive and dynamic stock investment prediction," in *Proc. AAAI Conf. Artif. Intell.*, 2024, pp. 14642–14650.
- [33] Y. Ma, R. Mao, Q. Lin, P. Wu, and E. Cambria, "Multi-source aggregated classification for stock price movement prediction," *Inf. Fusion*, vol. 91, pp. 515–528, 2023.
- [34] R. Sawhney, S. Agarwal, A. Wadhwa, T. Derr, and R. R. Shah, "Stock selection via spatiotemporal hypergraph attention network: A learning to rank approach," in *Proc. AAAI Conf. Artif. Intell.*, 2021, pp. 497–504.
- [35] Y.-L. Hsu, Y.-C. Tsai, and C.-T. Li, "FinGAT: Financial graph attention networks for recommending top-*k* profitable stocks," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 1, pp. 469–481, Jan. 2023.
- [36] P. Zhu, Y. Li, Y. Hu, Q. Liu, D. Cheng, and Y. Liang, "LSR-IGRU: Stock trend prediction based on long short-term relationships and improved GRU," in *Proc. 33rd ACM Int. Conf. Inf. Knowl. Manage.*, 2024, pp. 5135–5142.
- [37] P. Zhu, Y. Li, Q. Liu, D. Cheng, and C. Jiang, "Financial time series prediction with multi-granularity graph augmented learning," *IEEE Trans. Knowl. Data Eng.*, vol. 37, no. 11, pp. 6436–6449, Nov. 2025.
- [38] J. Yoo, Y. Soun, Y.-C. Park, and U. Kang, "Accurate multivariate stock movement prediction via data-axis transformer with multi-level contexts," in *Proc. 27th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2021, pp. 2037–2045.
- [39] Z. Cao, J. Xu, C. Dong, P. Yu, and T. Bai, "MATCC: A novel approach for robust stock price prediction incorporating market trends and cross-time correlations," in *Proc. 33rd ACM Int. Conf. Inf. Knowl. Manage.*, 2024, pp. 187–196.
- [40] D. B. Rubin, "Estimating causal effects of treatments in randomized and nonrandomized studies," *J. Educ. Psychol.*, vol. 66, no. 5, 1974, Art. no. 688.
- [41] J. Splawa-Neyman, D. M. Dabrowska, and T. P. Speed, "On the application of probability theory to agricultural experiments. essay on principles. section 9," *Stat. Sci.*, vol. 5, pp. 465–472, 1990.
- [42] S. R. Künzel, J. S. Sekhon, P. J. Bickel, and B. Yu, "Metalearners for estimating heterogeneous treatment effects using machine learning," *Proc. Nat. Acad. Sci.*, vol. 116, no. 10, pp. 4156–4165, 2019.
- [43] J. Pearl, *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*. Amsterdam, The Netherlands: Elsevier, 2014.
- [44] P. Spirtes, C. Glymour, and R. Scheines, *Causation, Prediction, and Search*. Cambridge, MA, USA: MIT Press, 2001.

- [45] D. M. Chickering, "Optimal structure identification with greedy search," *J. Mach. Learn. Res.*, vol. 3, no. Nov, pp. 507–554, 2002.
- [46] P. Hoyer, D. Janzing, J. M. Mooij, J. Peters, and B. Schölkopf, "Nonlinear causal discovery with additive noise models," in *Proc. Adv. Neural Inf. Process. Syst.*, 2008, pp. 689–696.
- [47] X. Zheng, B. Aragam, P. K. Ravikumar, and E. P. Xing, "DAGs with NO TEARS: Continuous optimization for structure learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2018, pp. 9492–9503.
- [48] A. Tank, I. Covert, N. Foti, A. Shojaie, and E. B. Fox, "Neural granger causality," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 8, pp. 4267–4279, Aug. 2022.
- [49] L. Schiatti, G. Nollo, G. Rossato, and L. Faes, "Extended granger causality: A new tool to identify the structure of physiological networks," *Physiol. Meas.*, vol. 36, no. 4, 2015, Art. no. 827.
- [50] H. Ye, E. R. Deyle, L. J. Gilarranz, and G. Sugihara, "Distinguishing time-delayed causal interactions using convergent cross mapping," *Sci. Rep.*, vol. 5, no. 1, 2015, Art. no. 14750.
- [51] Z. Benkő et al., "Complete inference of causal relations between dynamical systems," 2018, *arXiv: 1808.10806*.
- [52] G. Feng, K. Yu, Y. Wang, Y. Yuan, and P. M. Djurić, "Improving convergent cross mapping for causal discovery with gaussian processes," in *Proc. 2020 IEEE Int. Conf. Acoust., Speech Signal Process.*, 2020, pp. 3692–3696.
- [53] E. De Brouwer, A. Arany, J. Simm, and Y. Moreau, "Latent convergent cross mapping," in *Proc. Int. Conf. Learn. Representations*, 2020, pp. 1–12.
- [54] K. Butler, D. Waxman, and P. Djurić, "Tangent space causal inference: Leveraging vector fields for causal discovery in dynamical systems," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, pp. 120078–120102.
- [55] E. R. Deyle and G. Sugihara, "Generalized theorems for nonlinear state space reconstruction," *PLoS One*, vol. 6, no. 3, 2011, Art. no. e18295.
- [56] T. Sauer, J. A. Yorke, and M. Casdagli, "Embedology," *J. Stat. Phys.*, vol. 65, pp. 579–616, 1991.
- [57] F. Takens, "Detecting strange attractors in turbulence," in *Proc. Dyn. Syst. Turbulence*, Warwick, 1980, pp. 366–381.
- [58] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, Art. no. 30.
- [59] A. Zeng, M. Chen, L. Zhang, and Q. Xu, "Are transformers effective for time series forecasting," in *Proc. AAAI Conf. Artif. Intell.*, 2023, pp. 11121–11128.
- [60] P. Chen et al., "Pathformer: Multi-scale transformers with adaptive pathways for time series forecasting," in *Proc. Int. Conf. Learn. Representations*, 2024, pp. 1–19.
- [61] A. Mehrabian, E. Hoseinzade, M. Mazloum, and X. Chen, "Mamba meets financial markets: A graph-mamba approach for stock price prediction," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process.*, 2025, pp. 1–5.
- [62] B. W. Matthews, "Comparison of the predicted and observed secondary structure of t4 phage lysozyme," *Biochimica et Biophysica Acta (BBA)-Protein Struct.*, vol. 405, no. 2, pp. 442–451, 1975.
- [63] X. Glorot and Y. Bengio, "Understanding the difficulty of training deep feedforward neural networks," in *Proc. 13th Int. Conf. Artif. Intell. Statist. Workshop Conf. Proc.*, 2010, pp. 249–256.
- [64] F. X. Diebold and R. S. Mariano, "Comparing predictive accuracy," *J. Bus. Econ. Statist.*, vol. 20, no. 1, pp. 134–144, 2002.
- [65] Y. Liu et al., "Real-time topology estimation for active distribution system using graph-bank tracking Bayesian networks," *IEEE Trans. Ind. Inform.*, vol. 19, no. 4, pp. 6127–6137, Apr. 2023.
- [66] Y. Yuan et al., "Joint network topology inference via structural fusion regularization," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 10, pp. 10351–10364, Oct. 2023.



Haotian Liu received the BS degree in automation science and engineering from Xi'an Jiaotong University, Xi'an, China, in 2022. He is currently working toward the MS degree with the same institution. His research interests include graph learning, causal inference, data mining, and relevant applications.



Bowen Hu received the BS degree in automation science and engineering from Xi'an Jiaotong University, Xi'an, China, in 2019. He is currently working toward the PhD degree with the same institution. His research interests include data analysis and mining.



Yadong Zhou (Member, IEEE) received the BS and PhD degrees from Xi'an Jiaotong University, Xi'an, China, in 2004 and 2011, respectively. He was a post-doctoral researcher with the Chinese University of Hong Kong, Hong Kong, from 2014 to 2015. He is currently a professor with the Faculty of Electronic and Information Engineering, Xi'an Jiaotong University. His research interests include data analysis and mining, network science and its applications, and software defined networks. He has published more than 100 papers in prestigious refereed conferences and journals, such as *IEEE Transactions on Smart Grid*, *IEEE Transactions on Industrial Informatics*, *IEEE Transactions on Neural Networks and Learning Systems*, *IEEE Transactions on Automation Science and Engineering*, *IEEE INFOCOM*, *AAAI*, etc.



Yuxun Zhou received the BS degree in electrical engineering from Xi'an Jiaotong University, Xi'an, China, in 2009, the diplôme d'Ingenieur degree in applied mathematics from Ecole Centrale Paris, Gif-sur-Yvette, France, in 2012, and the PhD degree in electrical engineering from the Department of Electrical Engineering and Computer Sciences, University of California, Berkeley, CA, USA, in 2017. His research interests include artificial intelligence, statistical learning theory, and paradigms for modern information-rich, large-scale, and human-involved systems.